

Ascend C 高性能矢量算子编程实践

作者：赵佳敏

时间：2026年7月

目录

Part 1 昇腾矢量计算基础

Part 2 矢量算子编程实践

2.1 基础矢量编程入门

2.2 MemBase中高阶打磨

2.3 RegBase中高阶打磨

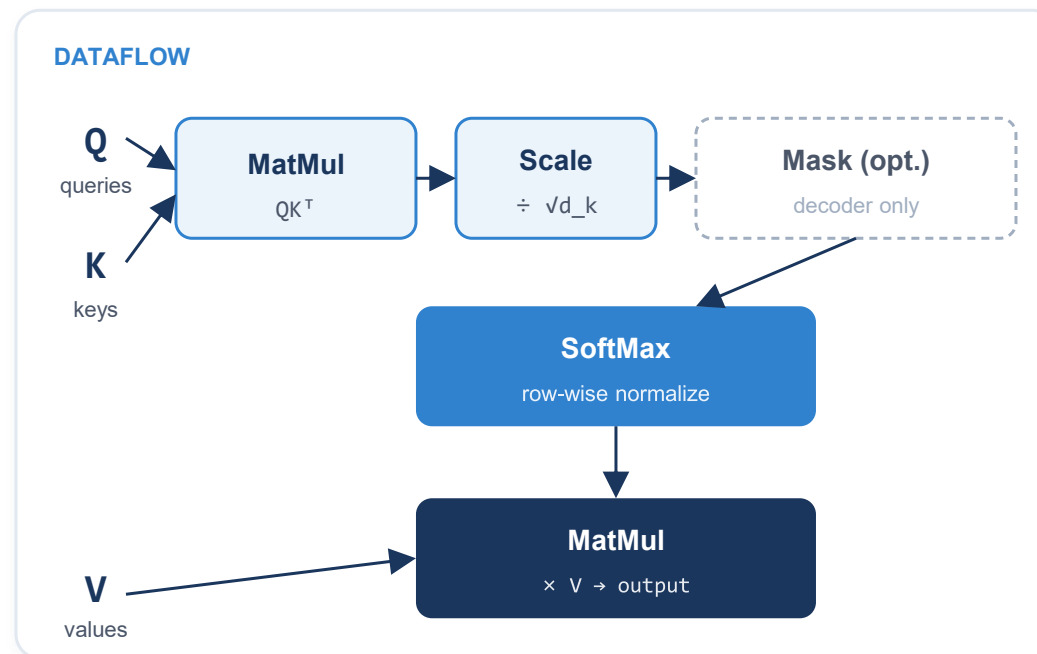
Part 3 能力全景与开发路径

01 昇腾矢量计算基础： 矢量运算的应用场景

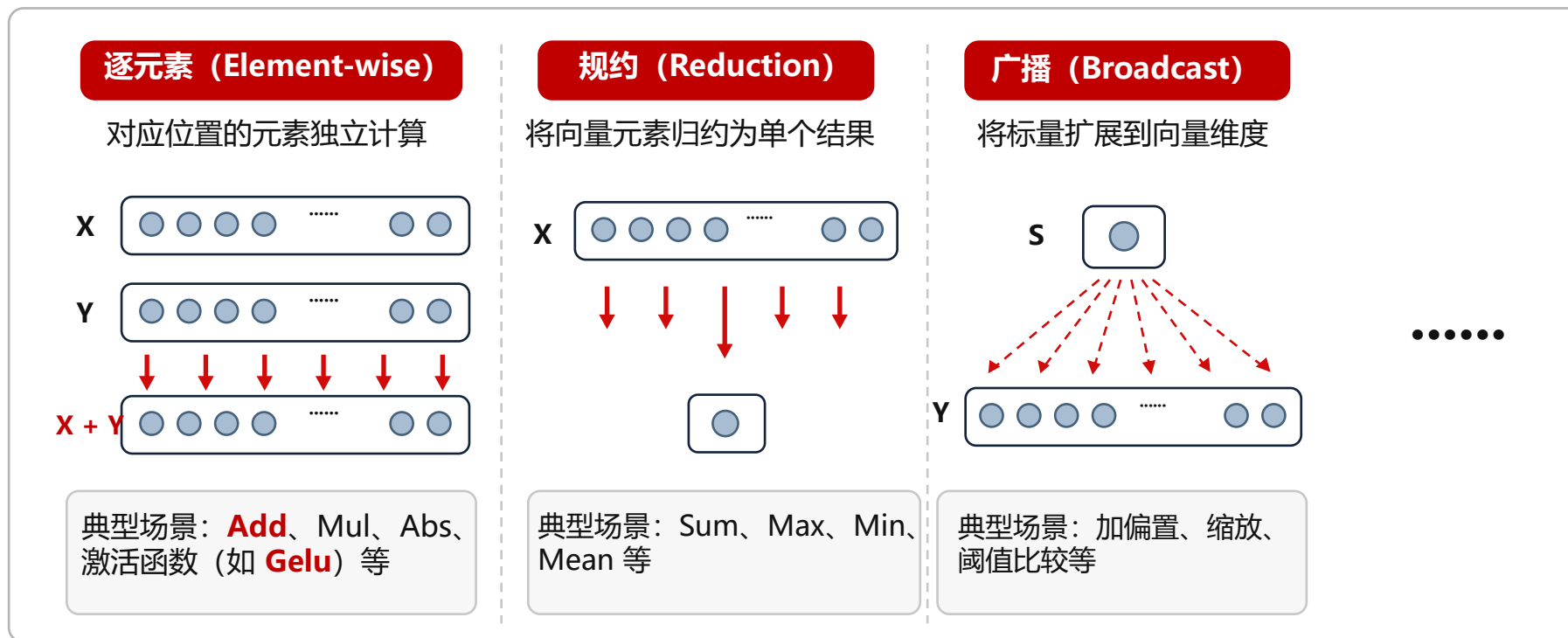
Attention Is All You Need

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

在这个公式中，分为矩阵运算和矢量运算两大部分。
其中，**Scale**、**SoftMax** 即由矢量计算单元承担。

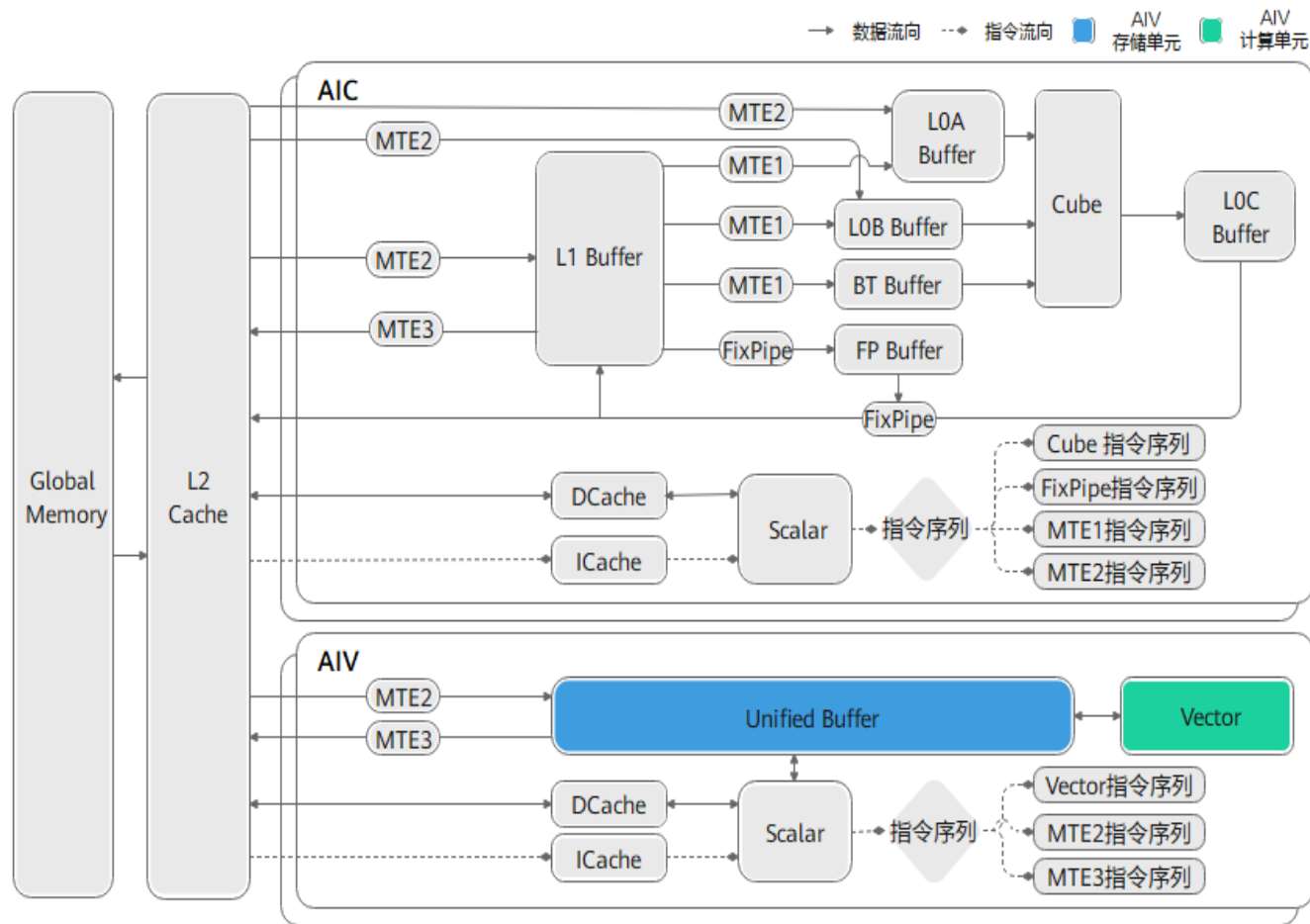


01 昇腾矢量计算基础： 矢量运算有哪些？



矢量计算是最基础、最通用的范式，也是我们本场的核心范围。

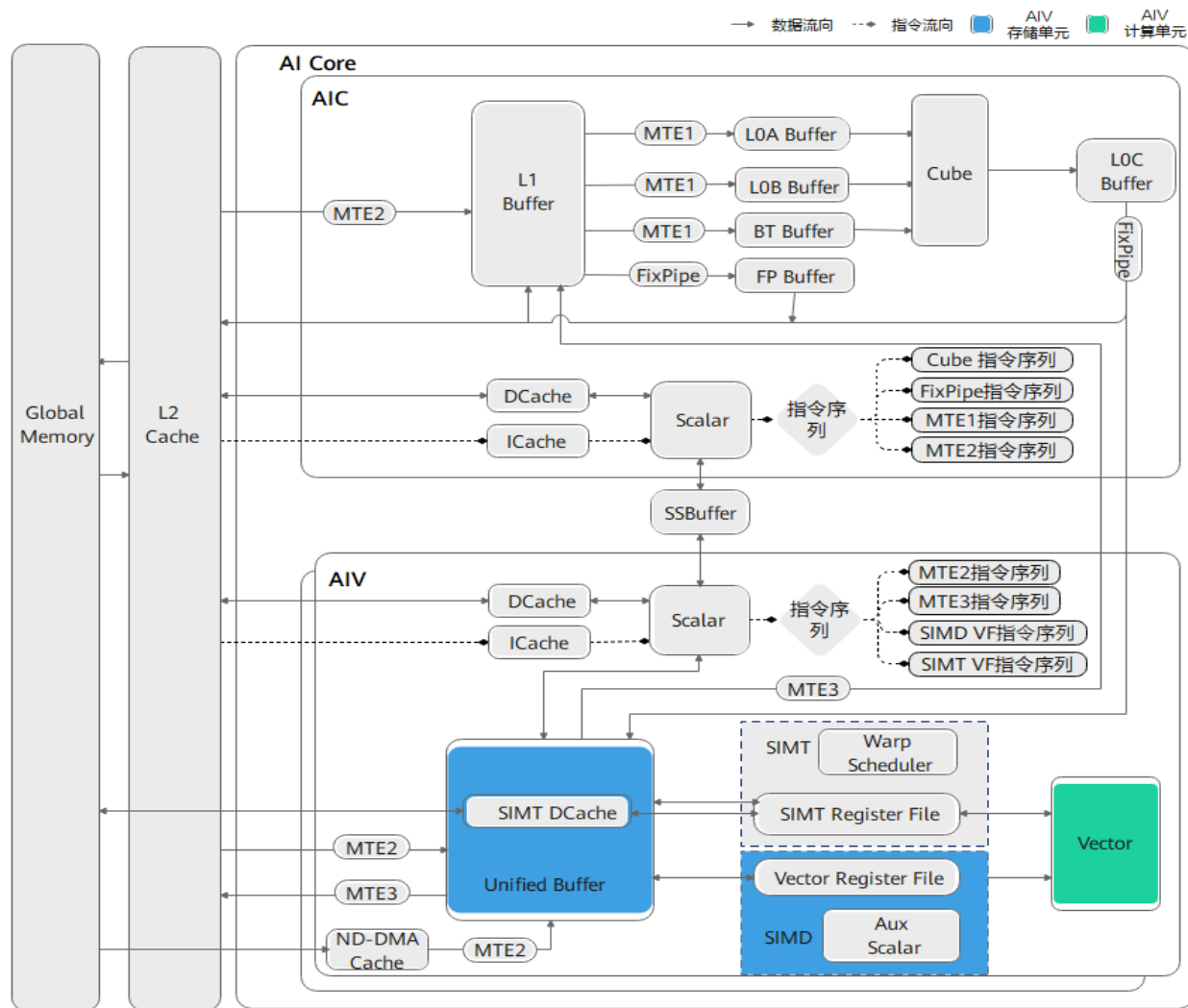
01 昇腾矢量计算基础： VECTOR逻辑架构-NPU架构版本2201



□ **Unified Buffer**: AI Core内部物理存储单元，通常用于存储向量计算的输入/输出数据。

□ **Vector**: Vector计算单元，专用于执行向量计算。

01 昇腾矢量计算基础： VECTOR逻辑架构-NPU架构版本3510



□ **Unified Buffer**: AI Core内部物理存储单元，通常用于存储向量计算的输入/输出数据。

□ **SIMD Register File**: AI Core内部物理存储单元，在SIMD程序中，数据从Unified Buffer搬运到Register进行计算，产生的中间结果可以不用传回Unified Buffer，直接在寄存器计算。

目录

Part 1 昇腾矢量计算基础

Part 2 矢量算子编程实践

2.1 基础矢量编程入门

2.2 MemBase中高阶打磨

2.3 RegBase中高阶打磨

Part 3 能力全景与开发路径

02 矢量算子编程实践：总体学习路径

同一套 Ascend C 矢量编程范式，逐级提升性能与工程能力

3 高阶

高阶：Add Case 5-6 + Gelu case2



Case 5-6
更大规模
并行与流水



Gelu
激活函数
完整实现



Reg/VF 融合
寄存器+向量
高阶打磨

映射到样例与手册

样例目录

https://gitcode.com/cann/asc-devkit/master/examples/01_simd_cpp_api/05_best_practices/02_reg_compute/gelu_high_performance

手册章节

<https://asc.gitcode.com/guide/编程指南/编程模型/AI-Core-SIMD编程/基于指针的C语言编程/Reg向量计算编程.html>

成长路径要点

• 入门：跑通 Add，建立范式。

• 中阶：Add Case 0-4，定位和优化瓶颈。

• 高阶：Add Case 5-6 + Gelu Reg case1-2。

2 中阶

中阶：Add Case 0-4 + Gelu case0-1



Case 0 基准
标量循环



Case 1 向量指令
一次处理整段数据



Case 2 多核并行
多 AI Core 协同



Case 3 搬运优化
增大 DataCopy 粒度



Case 4 双缓冲
搬运重叠

映射到样例与手册

样例目录

https://gitcode.com/cann/asc-devkit/master/examples/01_simd_cpp_api/05_best_practices/00_vector_compute/add_high_performance

手册章节

<https://asc.gitcode.com/guide/算子实践参考>

1 入门

入门：Introduction / Basic API



熟悉 Ascend C
编程模型与接口



跑通 Add
端到端链路



数据流转
GM ↔ UB ↔ AI Core



正确性验证
对齐 Host 结果

映射到样例与手册

样例目录

https://gitcode.com/cann/asc-devkit/master/examples/01_simd_cpp_api/00_introduction

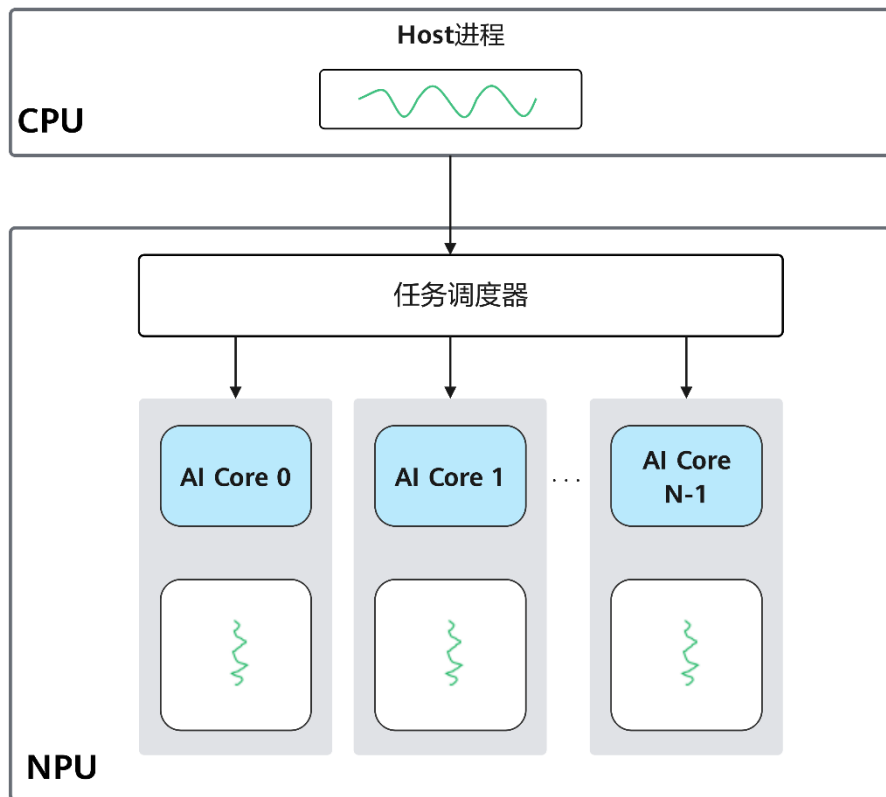
手册章节

<https://asc.gitcode.com/api/SIMD-API/基础API/Memory向量计算>

• 每层映射到样例目录和手册章节。

🎯 同一套矢量编程范式，标量 → 向量 → 多核 → 双缓冲搬运 → bank冲突 → Reg/VF，逐级打磨，持续逼近硬件上限。

02 矢量算子编程实践：异构编程模型



Host 端调用 (Ascend C/C++)

```
int main(int argc, char const* argv[])
{
    aclrtSetDevice(0); // 运行管理资源申请。
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);
    hello_world<<<8, 0, stream>>>();
    aclrtSynchronizeStream(stream);
    aclrtDestroyStream(stream);
    aclrtResetDevice(0); // 销毁运行资源。
    return 0;
}
```

Host侧主要负责**运行时管理**，包括存储、设备以及stream管理等，确保任务的高效调度；

HelloWorld 核函数 (Ascend C)

```
#include "utils/debug/asc_printf.h"
#include "acl/acl.h"

__global__ __vector__ void HelloWorld()
{
    printf( "Hello World!!!\n" );
}
```

Kernel核函数：完成批量数据的矢量运算、矩阵运算等计算密集型的任务。



手册地址
样例地址

<https://asc.gitcode.com/guide/编程指南/编程模型/异构系统.html>

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/00_introduction/00_quickstart/hello_world

CANN

02 矢量算子编程实践：MemoryBase 和 RegBase

MemBase vs RegBase对比：

维度	MemBase (基础API)	RegBase (VF融合API)
编程接口	Compute API (如 <code>Mul(yLocal, xLocal, xLocal, n)</code>)	Reg API (如 <code>Reg::Mul(yReg, xReg, xReg, mask)</code>)
数据驻留	每条指令隐式Load/Store, 中间结果写回UB	数据显式加载到寄存器后常驻, 中间结果暂存寄存器
同步方式	每条指令间需 <code>PipeBarrier<PIPE_V>()</code> 同步	VF函数内无需显式同步, 硬件自动管理依赖
适用场景	简单计算、单步或少步运算	多步骤融合计算, 减少UB读写次数
性能特征	每步Load/Store开销大, IPC低	仅1次Load+1次Store, 支持双发, IPC高

手册地址

- [1] <https://asc.gitcode.com/api/基础API/Memory矢量计算.html>
- [2] <https://asc.gitcode.com/api/基础API/Reg矢量计算.html>

Basic API样例介绍

概述

本样例介绍了Ascend C Basic API的使用方法，包括数据搬运、内存向量计算、Reg向量计算、矩阵计算、资源管理、同步控制、原子操作、TPipe/TQue和工具类API等。

样例列表

目录名称	功能描述
00_data_movement	本路径下包含了与数据搬运相关的多个API的样例，展示了数据搬运接口的使用。
01_memory_vector_compute	本样例路径以reduce、sort、transpose等算子为例，展示了内存向量计算接口的使用。
02_reg_vector_compute	本路径下包含了与向量计算相关的多个API的样例，基于Reg编程接口实现。
03_matrix_compute	本样例路径以batch_matmul等算子为例，展示了矩阵计算接口的使用。
04_memory_management	本路径下包含了与资源管理相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
05_sync_control	本路径下包含了与同步控制相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
06_atomic	本路径下包含了与原子操作相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
07_tpipe_tque	本路径下包含了与tpipe和tque相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
09_utils	本路径下包含了与工具相关API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。

样例地址

- [1] https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/01_memory_vector_compute
- [2] https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/03_basic_api/02_reg_vector_compute

02 矢量算子编程实践：实现一个最基础的Add (MemoryBase)

Ascend C 核函数示例 (Add - MemoryBase)

```
01 __vector__ __global__ void add_custom(__gm__ half* x, __gm__ half* y, __gm__ half* z)
02 {
03     AscendC::InitSocState();
04
05     AscendC::GlobalTensor<float> xGm, yGm, zGm;
06     xGm.SetGlobalBuffer(x + block_idx * blockLength, blockLength); 1
07     yGm.SetGlobalBuffer(y + block_idx * blockLength, blockLength);
08     zGm.SetGlobalBuffer(z + block_idx * blockLength, blockLength);
09
10     AscendC::LocalMemAllocator<AscendC::Hardware::UB> ubAllocator;
11     AscendC::LocalTensor<float> xLocal = ubAllocator.Alloc<float, blockLength>(); 2
12     AscendC::LocalTensor<float> yLocal = ubAllocator.Alloc<float, blockLength>();
13     AscendC::LocalTensor<float> zLocal = ubAllocator.Alloc<float, blockLength>();
14
15     AscendC::DataCopy(xLocal, xGm, blockLength); 3
16     AscendC::DataCopy(yLocal, yGm, blockLength);
17     AscendC::PipeBarrier<PIPE_ALL>();
18
19     AscendC::Add(zLocal, xLocal, yLocal, blockLength); 4
20     AscendC::PipeBarrier<PIPE_ALL>();
21
22     AscendC::DataCopy(zGm, zLocal, blockLength); 5
23     AscendC::PipeBarrier<PIPE_ALL>();
24 }
```

1



SetGlobalBuffer: 按 block_idx 做 GM 偏移

- 计算每个核负责的元素区间。
- 通过 offset 指定位于 GM 中的起始位置。

2



UB 分配: 在 UB 分配输入输出 Tensor

- 为 x、y、z 申请 UB 空间。

3



DataCopy 搬入: GM -> UB (MTE2流水)

- DataCopy 将输入从 GM 搬入 UB。
- 使用 PipeBarrier 等待搬入完成再开始计算。

4



计算: 相加 (VEC 流水)

- 使用 AscendC::Add 相加。
- 使用 PipeBarrier 等待计算完成再开始搬运。

5



DataCopy 搬回: UB -> GM (MTE3流水)

- DataCopy 将结果从 UB 搬回 GM。
- PipeBarrier<PIPE_ALL>() 等待搬回完成。

02 矢量算子编程实践：实现一个最基础的Gelu (RegBase)

Ascend C 核函数示例 (Gelu - RegBase)

```
01 _simd_vf_ inline static void GeluVfMethod2(  
02   __ubuf__ float* xAddr, __ubuf__ float* yAddr, uint32_t count, uint32_t loopNum)  
03 {  
04   // oneRepeatSize 表示一次向量重复能处理多少个 float 元素。  
05   constexpr uint32_t oneRepeatSize = AscendC::GetVecLen() / sizeof(float);  
06   AscendC::Reg::MaskReg mask;  
07   AscendC::Reg::RegTensor<float> xReg;  
08   AscendC::Reg::RegTensor<float> yReg;  
09   AscendC::Reg::RegTensor<float> tmpReg;  
10   for (uint32_t i = 0; i < loopNum; ++i) {  
11     mask = AscendC::Reg::UpdateMask<float>(count);  
12     // 从 UB 中取一段数据到寄存器。  
13     AscendC::Reg::LoadAlign(xReg, xAddr + i * oneRepeatSize);  
14     AscendC::Reg::Mul(yReg, xReg, xReg, mask); // x^2  
15     AscendC::Reg::Mul(yReg, yReg, xReg, mask); // x^3  
16     AscendC::Reg::Muls(yReg, yReg, COEFF_CUBIC, mask); // -0.071405 * x^3  
17     AscendC::Reg::Muls(tmpReg, xReg, COEFF_LINEAR, mask); // -1.595769 * x  
18     AscendC::Reg::Add(yReg, tmpReg, yReg, mask); // -1.595769 * x - 0.071405 * x^3  
19     AscendC::Reg::Exp(yReg, yReg, mask); // exp(...)  
20     AscendC::Reg::Adds(yReg, yReg, 1.0f, mask); // 1 + exp(...)  
21     AscendC::Reg::Div(yReg, xReg, yReg, mask); // x / (1 + exp(...))  
22     AscendC::Reg::StoreAlign(yAddr + i * oneRepeatSize, yReg, mask);  
23   }
```

4

1



SetGlobalBuffer: 按 block_idx 做 GM 偏移

- 计算每个核负责的元素区间。
- 通过 offset 指定位于 GM 中的起始位置。

2



UB 分配: 在 UB 分配输入输出 Tensor

- 为 x、y、z 申请 UB 空间。

3



DataCopy 搬入: GM -> UB (MTE2流水)

- DataCopy 将输入从 GM 搬入 UB。
- 使用 set/wait MTE2_V 等待搬入完成再开始计算。

4



VF 计算, 搬入 REG 寄存器中进行计算

- 通过 asc_vf_call 调用 GeluVF 函数
- 在 VF 函数内完成 LoadAlign -> Mul.. -> StoreAlign

5



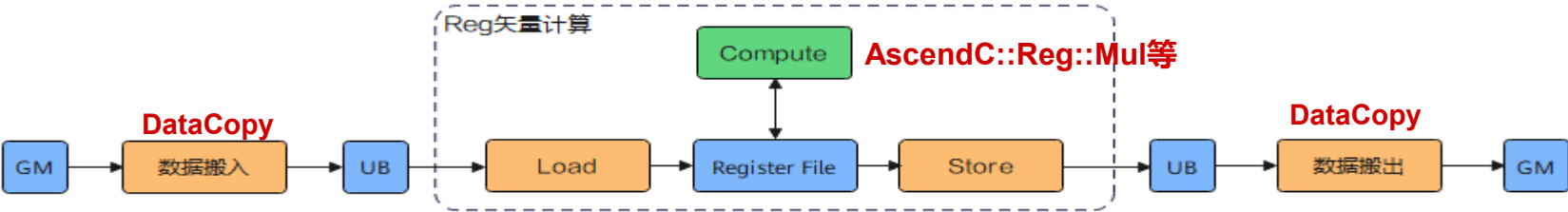
DataCopy 搬回: UB -> GM (MTE3流水)

- DataCopy 将结果从 UB 搬回 GM。
- PipeBarrier<PIPE_ALL>() 等待搬回完成。

02 向量算子编程实践：如何使用手册和样例编写一个VECTOR算子



手册地址	样例地址
DataCopy https://asc.gitcode.com/api/SIMD-API/基础API/数据搬运/数据搬运/GM与UB数据搬运.html	https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/03_basic_api/00_data_movement/
PipeBarrier https://asc.gitcode.com/api/SIMD-API/基础API/同步控制/核内同步/PipeBarrier(ISASI).html	https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/03_basic_api/05_sync_control/
set/wait https://asc.gitcode.com/api/SIMD-API/基础API/同步控制/核内同步/SetFlag-WaitFlag.html	
asc_vf_call https://asc.gitcode.com/api/SIMD-API/基础API/Reg向量计算/VF调用/asc_vf_call.html	
Reg::LoadAlign Reg::StoreAlign https://asc.gitcode.com/api/SIMD-API/基础API/基础API列表/Reg向量计算.html#Reg数据搬运	[1] https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/00_introduction/04_reg_compute/add [2] https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/00_introduction/04_reg_compute/gelu
Reg::Add https://asc.gitcode.com/api/SIMD-API/基础API/Reg向量计算/基础算术/Add.html	



02 矢量算子编程实践：如何使用手册和样例编写一个VECTOR算子

Step1 查阅API手册

Ascend C

1. Ascend C API列表

2. SIMD API

2.1. 通用说明和约束

2.2. 基础API

2.2.1. 基础API列表

2.2.2. 数据结构

2.2.3. 数据搬运导览

2.2.4. 矩阵计算 (ISASI)

2.2.5. 矩阵计算 (Tensor API)

2.2.6. Memory向量计算

2.2.7. Reg向量计算

2.2.8. 标量计算

2.2.9. 资源管理

2.2.10. 同步控制

2.2.11. 缓存控制

2.2.12. 原子操作

2.2.13. 调试接口

2.2.14. 工具接口

2.2.15. Cube分组管理 (ISASI)

2.2.16. Kernel Tiling

2.2.17. 特殊寄存器访问

2.3. C API

2.4. 高阶API

<https://asc.gitcode.com/api/SIMD-API/基础API.html>

Step2 查看Basic API 样例

Basic API样例介绍

概述

本样例介绍了Ascend C Basic API的使用方法，包括数据搬运、内存向量计算、Reg向量计算、矩阵计算、资源管理、同步控制、原子操作、TPipe/TQue和工具类API等。

样例列表

目录名称	功能描述
00_data_movement	本路径下包含了与数据搬运相关的多个API的样例，展示了数据搬运接口的使用。
01_memory_vector_compute	本样例路径以reduce、sort、transpose等算子为例，展示了内存向量计算接口的使用。
02_reg_vector_compute	本路径下包含了与向量计算相关的多个API的样例，基于Reg编程接口实现。
03_matrix_compute	本样例路径以batch_matmul等算子为例，展示了矩阵计算接口的使用。
04_memory_management	本路径下包含了与资源管理相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
05_sync_control	本路径下包含了与同步控制相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
06_atomic	本路径下包含了与原子操作相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
07_tpipe_tque	本路径下包含了与tpipe和tque相关的多个API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。
09_utils	本路径下包含了与工具相关API的样例。每个样例均基于Ascend C的<<<>>>直调方法，支持main函数和kernel函数在同一个cpp文件中实现。

https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/03_basic_api

目录

Part 1 昇腾矢量计算基础

Part 2 矢量算子编程实践

2.1 基础矢量编程入门

2.2 MemBase中高阶打磨

2.3 RegBase中高阶打磨

Part 3 能力全景与开发路径

02 中级性能优化: 如何定位性能瓶颈

Profiling 关键指标解释 (以 msprof 为例)

指标	含义	典型来源	越高越好/越低越好
 Task Duration (μs)	该 Task 在 AI Core 上总耗时 (微秒)	AI Core 执行时间	越低越好
 aiv_vec_time (μs)	AIV Core 计算指令执行时间	向量计算	越低越好
 aiv_mte2_time (μs)	从 GM 读取到 UB 的搬运时间 (读搬运)	DataCopy (GM→UB)	越低越好
 aiv_mte3_time (μs)	从 UB 写回 GM 的搬运时间 (写搬运)	DataCopy (UB→GM)	越低越好



占比口径:
占比 = 该指标耗时 / Task Duration × 100%

用 `msprof ./demo` 抓取性能数据。

瓶颈定位思路



查看各指标
耗时与占比

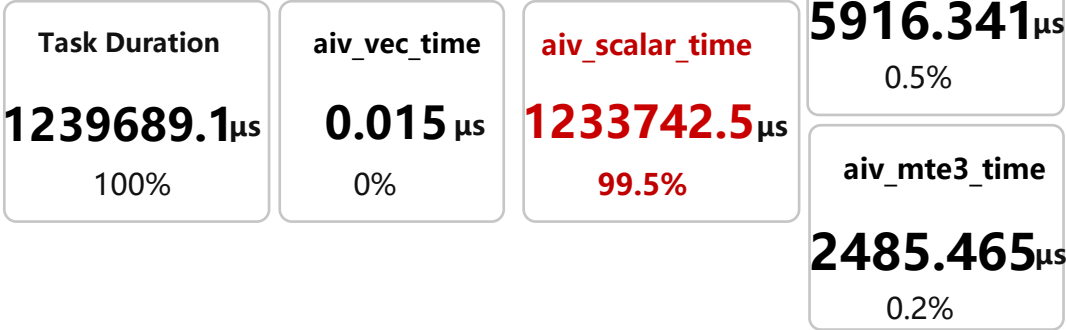


定位和理论数值
差异较大的地方、
耗时较长的流水



该指标类型
即为瓶颈

占比示例 (以Add最佳实践的case0为例)



结论: 标量运算 (aiv_scalar) 占比最高, 为当前瓶颈



1 关注 Task Duration、aiv_vec_time、aiv_mte2、aiv_mte3等。



2 计算、读搬运、写搬运占比能指向瓶颈。



3 谁占比较高, 就优先优化谁。

手册地址

<https://asc.gitcode.com/guide/算子实践参考/性能分析.html>

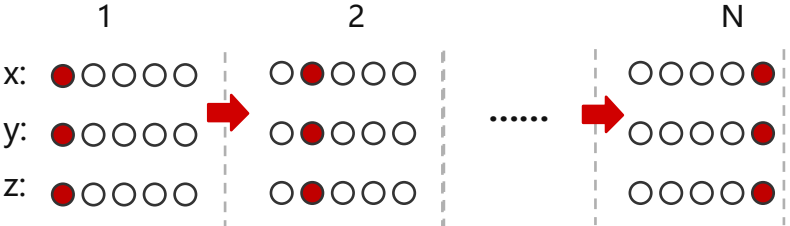
场景: 8192×8192 half 矩阵相加 | 测试平台: Atlas A2 训练系列

02 中级性能优化: ① 别用标量循环算向量

Case 0 基准: 标量循环

```
for (...) {  
    z = GetValue(x) + GetValue(y);  
    SetValue(z);  
}
```

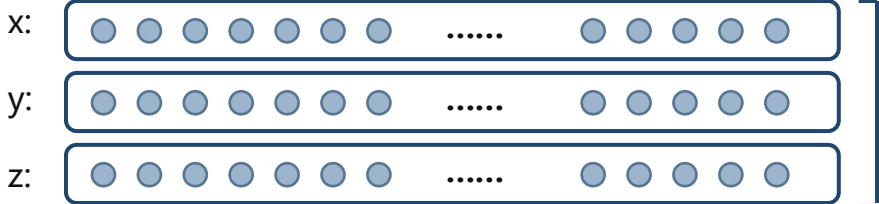
逐元素串行处理 (一次处理一个)



Case 1: 向量指令

AscendC::Add(zLocal, xLocal, yLocal, len);
一条指令处理整段数据

向量并行处理 (一次处理多个)



一条指令
并行完成

	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case0	1239689.1	1239688.6	0.015	0	1233742.5	0.995	5916.341	0.005	2485.465	0.002
Case1	6909.6	6909.14	761.65	0.11	231.166	0.033	6208.762	0.873	2613.205	0.378

提升99.4%



样例地址

https://gitcode.com/cann/asc-devkit/blob/master/examples/01_simd_cpp_api/05_best_practices/00_vector_compute/add_high_performance

场景: 8192×8192 half 矩阵相加 | 测试平台: Atlas A2 训练系列

02 中级性能优化 ② 采用多核均匀切分

Case 1: 单核处理 (行方向)

```
add_custom<...>
  <<<1, 0, stream>>>(...);
```

行方向 (单核处理全部行)

Core 0

x: [.....]

y: [.....]

z: [.....]

AI Core 数: 1

Case 2: 多核均匀切分 (行方向)

```
add_custom<...>
  <<<48, 0, stream>>>(...);
```

行方向 (按行均匀切分给多个 Core)

Core 0

x: [.....]

y: [.....]

Core 1

x: [.....]

y: [.....]

Core 2

x: [.....]

y: [.....]

.....

Core 47

x: [.....]

y: [.....]

各核独立工作
互不干扰

AI Core 数: 48

切分示意 (按行切分)

GM

M 行

AI Core

Core 0

Core 1

Core 2

.....

Core 47

余数分给前几个核

手册地址

<https://asc.gitcode.com/guide/算子实践参考/SIMD算子实现/矢量编程/多核-Tiling切分/多核Tiling.html>

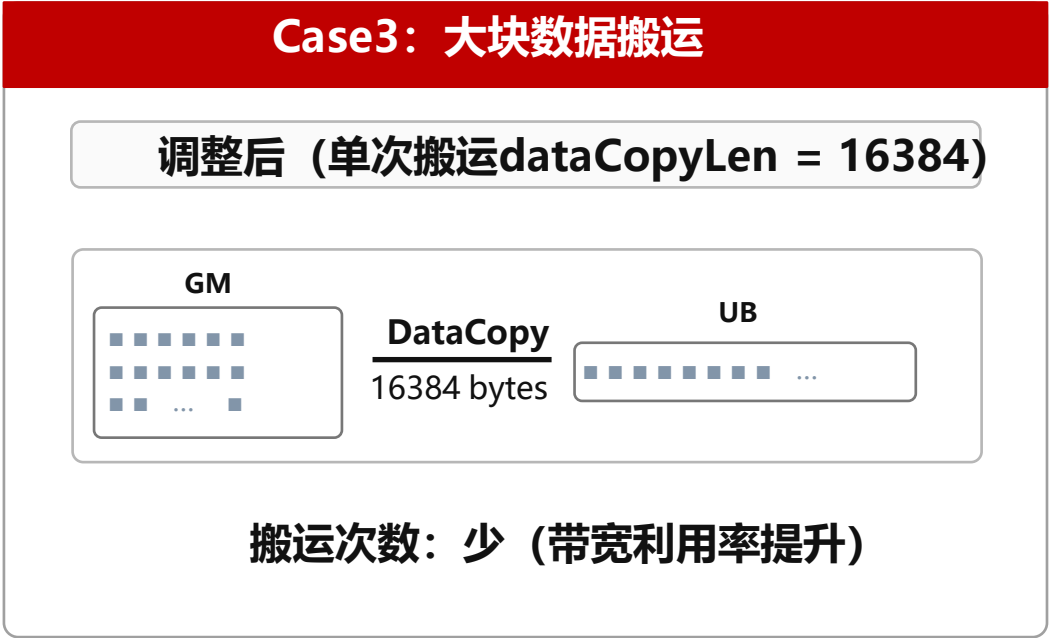
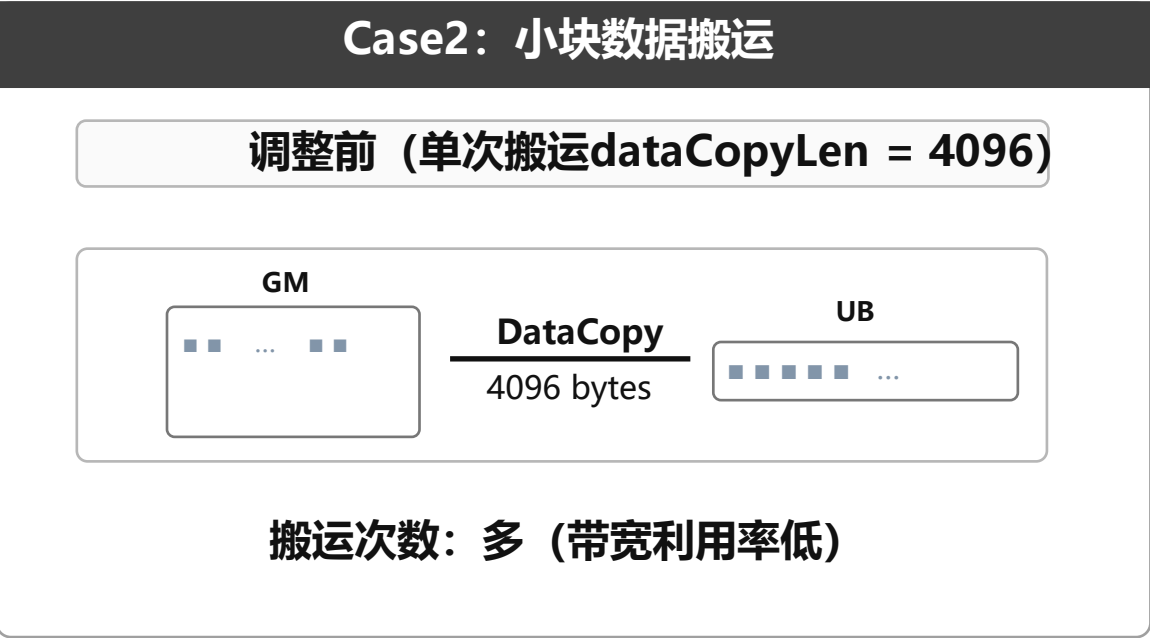
	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case1	6909.6	6909.14	761.65	0.11	231.166	0.033	6208.762	0.873	2613.205	0.378
Case2	306.58	297.14	15.885	0.053	8.219	0.028	215.485	0.725	54.33	0.183

总耗时再降
95.5%

$$BW_{read} = \frac{D_{read}}{T_{mte2}} = \frac{8192 \times 8192 \times 2 \times 2}{215.485 \times 10^{-6}} = 1.2457 \text{ TB/S}$$

mte2搬运 (aiv_mte2_ratio) 占比最高, 带宽利用率仍然不足

02 中级性能优化 ③ 增大搬运粒度

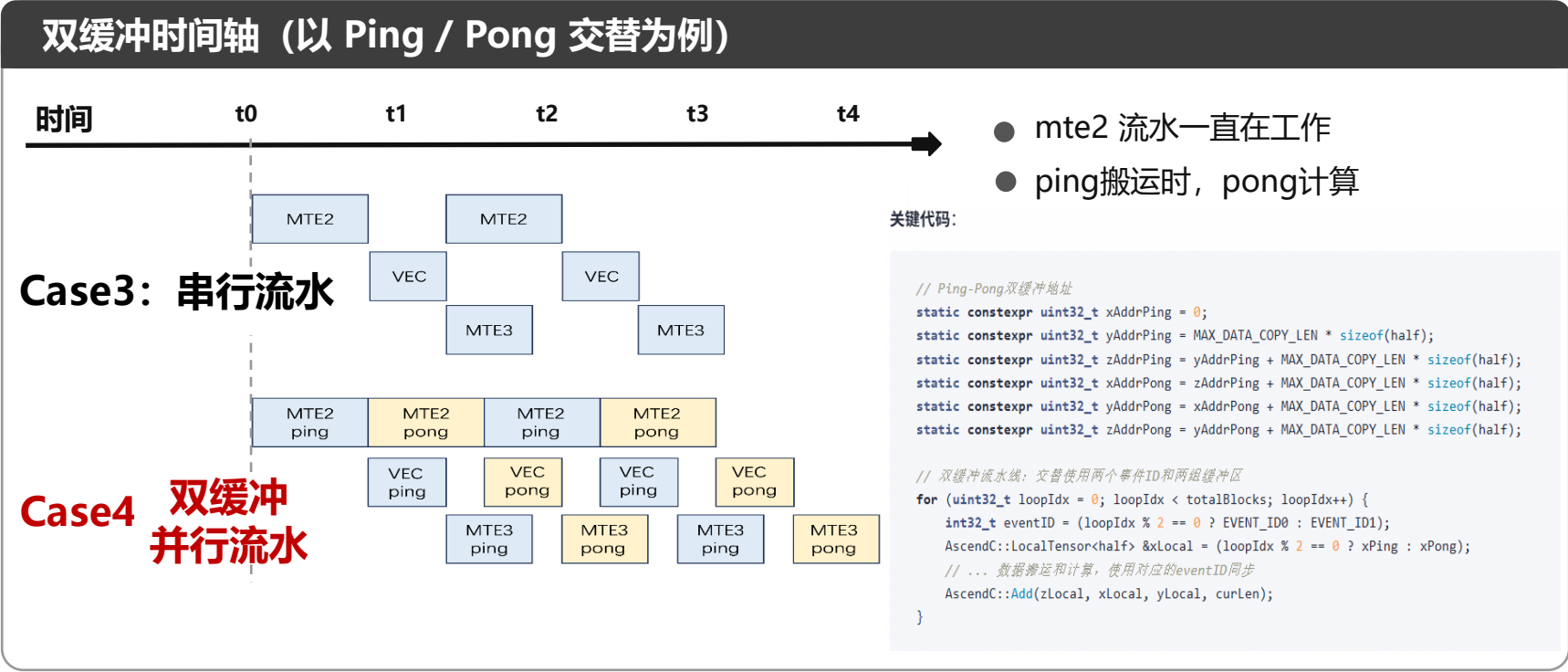


	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case2	306.58	297.14	15.885	0.053	8.219	0.028	215.485	0.725	54.33	0.183
Case3	268.5	261.5	12.845	0.049	2.833	0.011	184.369	0.705	57.354	0.219

总耗时再降
12.4%

$$BW_{read} = \frac{D_{read}}{T_{mte2}} = \frac{8192 \times 8192 \times 2 \times 2}{184.369 \times 10^{-6}} = 1.4560 \text{ TB/S}$$

02 中级性能优化 ④ 双缓冲流水并行



	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case3	268.5	261.5	12.845	0.049	2.833	0.011	184.369	0.705	57.354	0.219
Case4	264.02	257.56	12.846	0.05	2.796	0.011	250.528	0.973	84.988	0.33

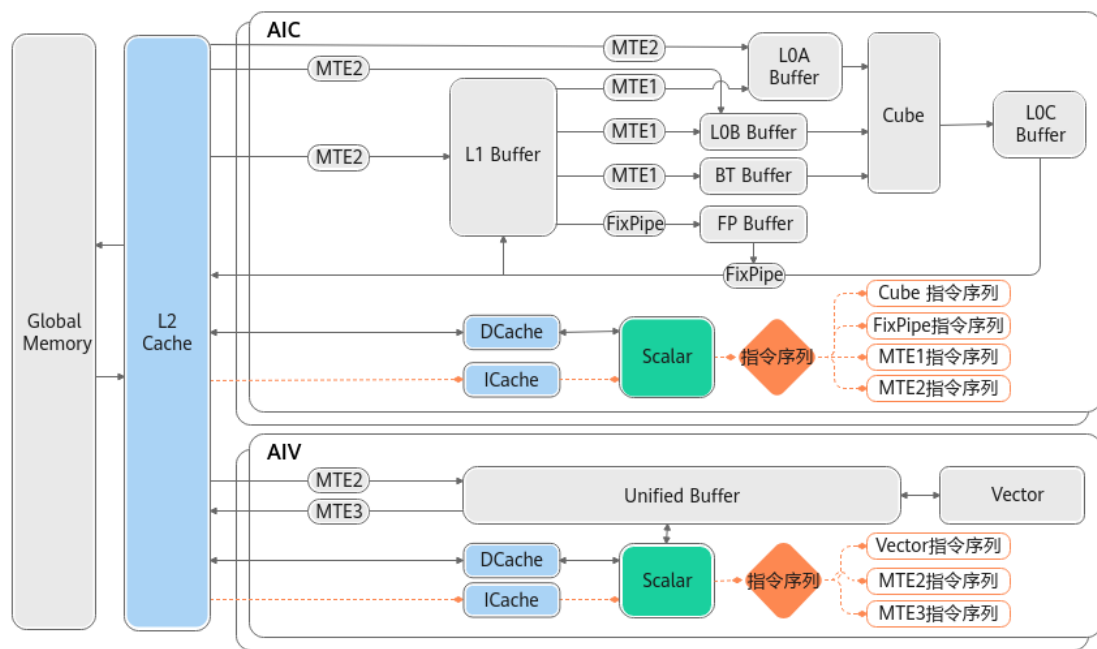
总耗时再降
1.67%

读写混合带宽: 开启双缓冲后, 读写流水是并行的, 因此采用读写混合带宽来计算

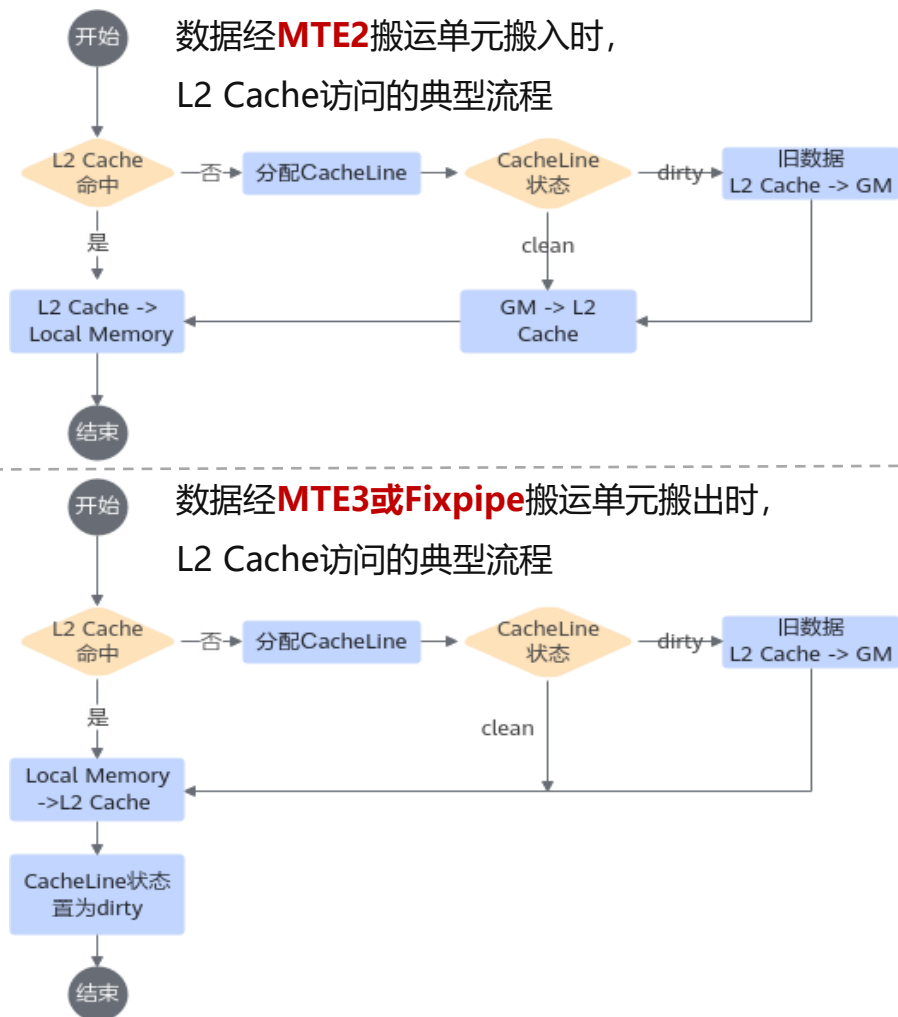
$$BW_{rw} = \frac{D_{read} + D_{write}}{T_{mte2}} = \frac{8192 \times 8192 \times (2 + 1) \times 2}{250.528 \times 10^{-6}} = 1.6072 \text{ TB/S}$$

02 高阶性能打磨 ⑤ L2 cache bypass

什么是 L2 Cache ?



L2 Cache 的访问原理



02 高阶性能打磨 ⑤ L2 cache bypass

- Add 输入只读一次，属于流式访问。
- 默认走 L2 可能污染 Cache。
- 对 xGm/yGm 设置 SetL2CacheHint(CACHE_MODE_DISABLE)。

关键代码：

```
xGm.SetL2CacheHint(AscendC::CacheMode::CACHE_MODE_DISABLE);  
yGm.SetL2CacheHint(AscendC::CacheMode::CACHE_MODE_DISABLE);
```

	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case4	264.02	257.56	12.846	0.05	2.796	0.011	250.528	0.973	84.988	0.33
Case5	187.68	185.15	12.846	0.069	4.479	0.024	175.997	0.951	160.291	0.866

总耗时下降

28.9%

理论VECTOR耗时：

$$T_{\text{theory}} = \frac{M \times N}{128 \times 1.85 \times 10^9 \times \text{核数}}$$

$$T_{\text{theory}} = \frac{8192 \times 8192}{128 \times 1.85 \times 10^9 \times 48} = \frac{67108864}{1.13664 \times 10^{13}} \approx 5.904 \times 10^{-6} \text{ s} = 5.904 \mu\text{s}$$

向量指令耗时仍然有优化空间

读写混合带宽：

$$BW_{rw} = \frac{D_{read} + D_{write}}{T_{mte2}} = \frac{8192 \times 8192 \times (2 + 1) \times 2}{175.997 \times 10^{-6}} = 2.2755 \text{ TB/S}$$

02 高阶性能打磨 ⑥ 避免UB Bank Conflict

UB BANK 结构 – 2201架构



向量指令一拍读 256B。



读写冲突：

读操作和写操作同时尝试访问同一个bank。



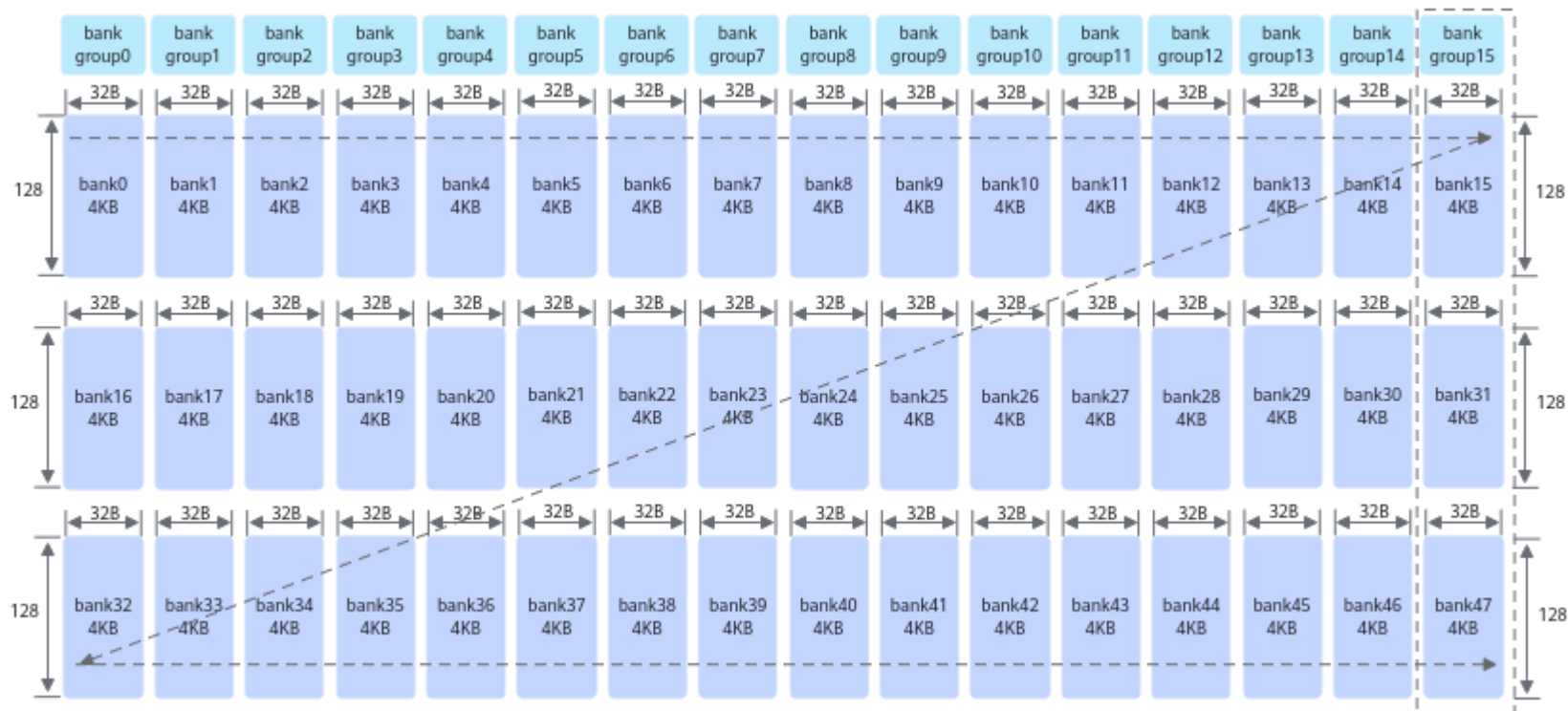
写写冲突：

多个写操作同时尝试访问同一个bank group。



读读冲突：

多个读操作同时尝试访问同一个bank group。



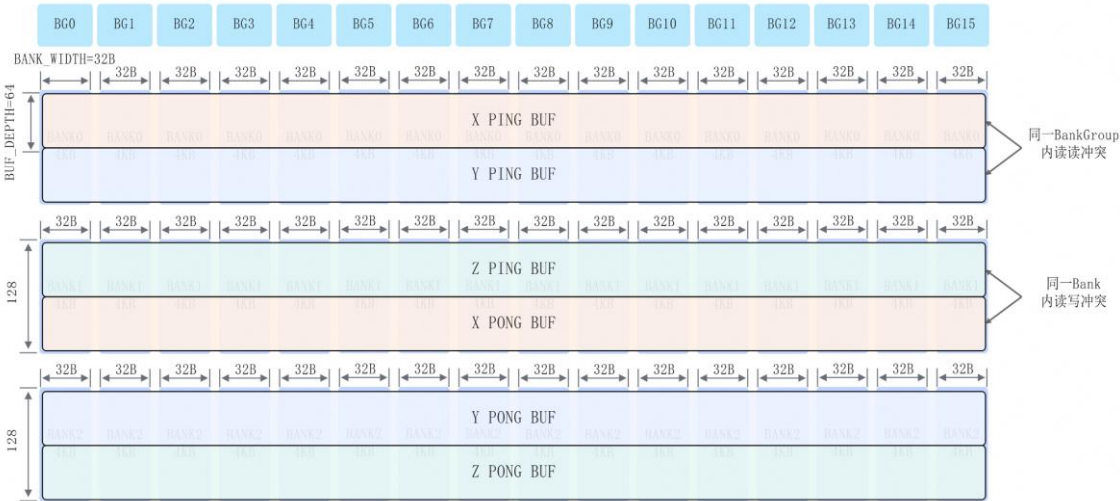
手册地址

<https://asc.gitcode.com/guide/算子实践参考/SIMD算子性能优化/内存访问/避免UB的bank冲突/避免bank冲突（NPU架构版本2201）.html>

02 高阶性能打磨 ⑥ 避免UB Bank Conflict

Case5: Bank 冲突 (同 Bank/Bank Group)

dataCopyLen = 16384 (16KB)

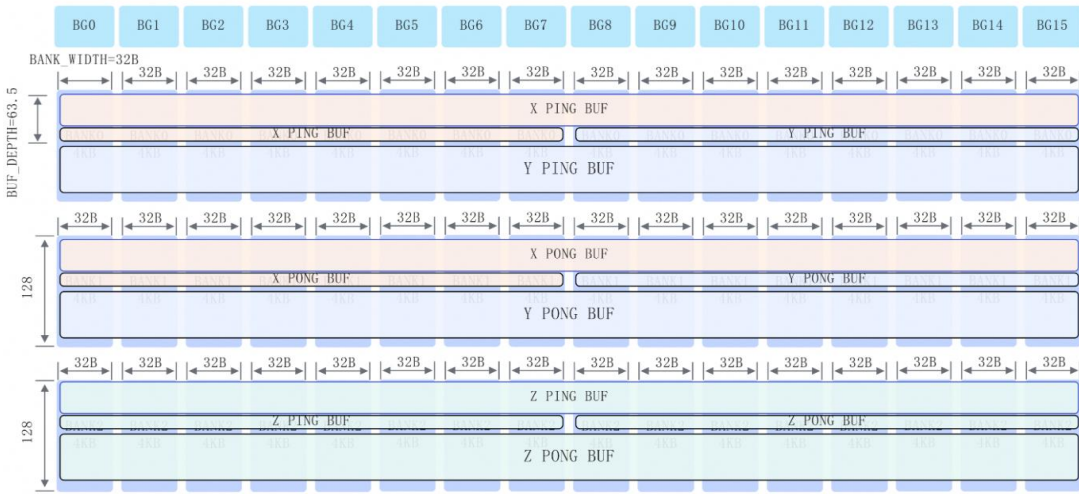


多份数据起始地址落在同一 Bank/Bank Group, 向量读 256B 时发生 Bank Conflict, 指令串行运行, 造成性能下降。



case6: 错开地址 (无冲突)

dataCopyLen = 16256 (15.875KB)



起始地址错开, 不同数据落在不同 Bank, 向量一拍读 256B 时并行访问, 无 Bank Conflict, 充分发挥带宽, 性能提升。

	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case5	187.68	185.15	12.846	0.069	4.479	0.024	175.997	0.951	160.291	0.866
Case6	184.52	178.49	6.954	0.039	3.424	0.019	171.611	0.961	121.442	0.68

目录

Part 1 昇腾矢量计算基础

Part 2 矢量算子编程实践

2.1 基础矢量编程入门

2.2 MemBase中高阶打磨

2.3 RegBase中高阶打磨

Part 3 能力全景与开发路径

02 中级性能优化 ① VF融合

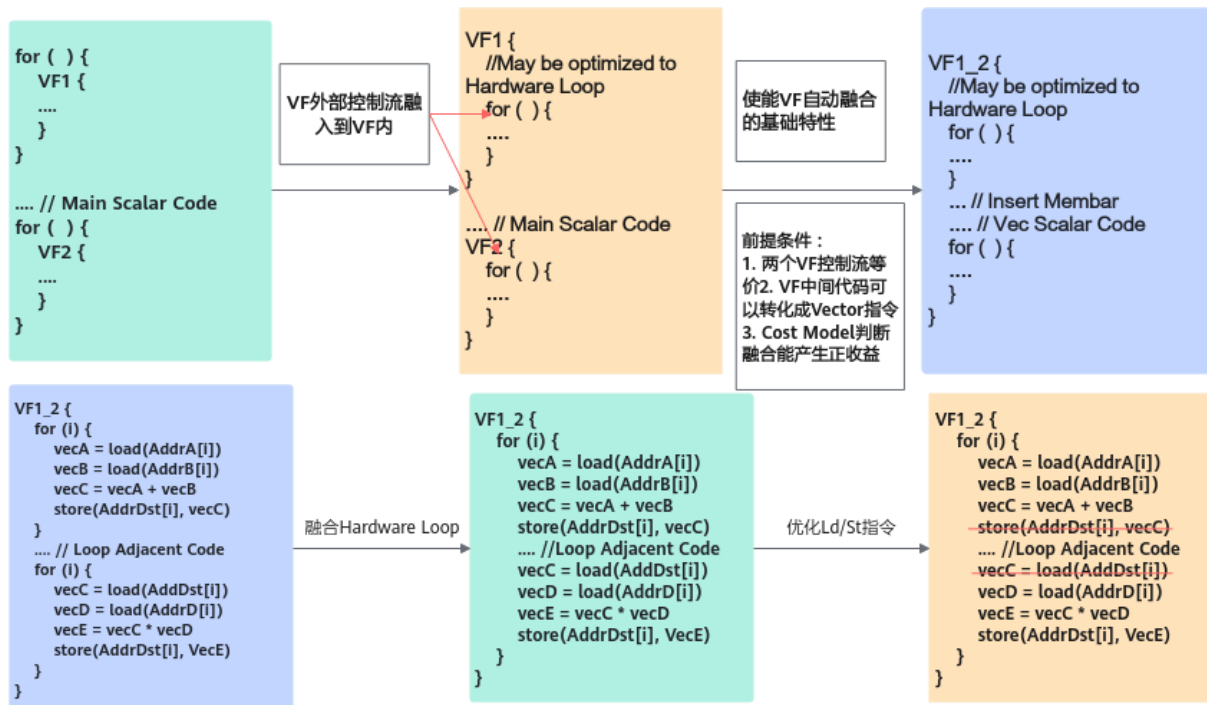
什么是VF融合？

VF函数：Reg矢量计算必须统一放到一个函数中，此函数定义为SIMD VF（Vector Function），需要通过`asc_vf_call`接口调用。

VF融合是将代码中**多个VF函数融合成一个VF函数**，有效提升性能。VF融合特性是优化特性，VF自动融合会借助Loop Fuse算法，将VF转换成Loop形态，然后将控制流等价（Control-Flow-Equivalent）的VF进行融合，最后将VF进行还原。

VF融合带来的收益：

- 消除冗余VF启动开销；
- 冗余Load/Store指令对消；
- Hardware Loop融合；



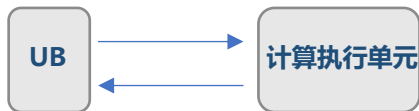
02 中级性能优化 ① VF融合

MemBase (基础API)

```
__aicore__ inline void GeluCompute(  
    const AscendC::LocalTensor<float>& xLocal, const AscendC::LocalTensor<float>& yLocal, uint32_t n)  
{  
    // yLocal = x * x = x^2  
    AscendC::Mul(yLocal, xLocal, xLocal, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = x^2 * x = x^3  
    AscendC::Mul(yLocal, yLocal, xLocal, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = x^3 * 0.044715 = 0.044715 * x^3  
    AscendC::Muls(yLocal, yLocal, COEFF_A, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = x + 0.044715 * x^3  
    AscendC::Add(yLocal, xLocal, yLocal, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = -1.595769 * (x + 0.044715 * x^3)  
    AscendC::Muls(yLocal, yLocal, COEFF_B, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = e^(-1.595769 * (x + 0.044715 * x^3))  
    AscendC::Exp(yLocal, yLocal, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = 1 + e^(-1.595769 * (x + 0.044715 * x^3))  
    AscendC::Adds(yLocal, yLocal, (float)1.0, n);  
    AscendC::PipeBarrier<PIPE_V>();  
    // yLocal = x / (1 + e^(-1.595769 * (x + 0.044715 * x^3)))  
    AscendC::Div(yLocal, xLocal, yLocal, n);  
}
```

aiv_vec_time
147.9 μ s

数据通路 (每步都在 UB 与 执行单元间来回搬)



特点

- **UB读写带宽被反复占用**: 中间结果频繁进出 UB, 搬运次数多
- 计算链长时性能受限, 对UB的反复读写极大**增加bank冲突的概率**
- 适合快速验证功能正确性

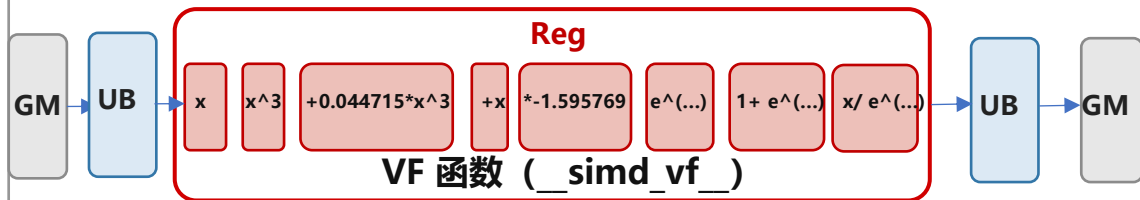
RegBase (RegBase API + VF融合)

```
// Case 1: RegBase API + VF融合, 寄存器内完成全部计算  
__simd_vf__ inline void GeluVfBasic(__ubuf__ float* xAddr, __ubuf__ float* yAddr, uint32_t n, uint32_t loopNum)  
{  
    constexpr uint32_t oneRepeatSize = AscendC::GetVecLen() / sizeof(float);  
    AscendC::Reg::MaskReg mask;  
    AscendC::Reg::RegTensor<float> xReg, yReg;  
  
    for (uint16_t i = 0; i < loopNum; ++i) {  
        mask = AscendC::Reg::UpdateMask<float>(n);  
        AscendC::Reg::LoadAlign(xReg, xAddr + i * oneRepeatSize); // 仅一次load  
        AscendC::Reg::Mul(yReg, xReg, xReg, mask);  
        AscendC::Reg::Muls(yReg, yReg, xReg, mask);  
        AscendC::Reg::Adds(yReg, yReg, COEFF_A, mask);  
        AscendC::Reg::Muls(yReg, yReg, COEFF_B, mask);  
        AscendC::Reg::Exp(yReg, yReg, mask);  
        AscendC::Reg::Adds(yReg, yReg, 1.0f, mask);  
        AscendC::Reg::Div(yReg, xReg, yReg, mask);  
        AscendC::Reg::StoreAlign(yAddr + i * oneRepeatSize, yReg, mask); // 仅一次store  
    }  
}
```

aiv_vec_time
66.277 μ s

向量计算指令
耗时下降
55.2%

数据通路 (中间结果常驻寄存器)



特点

- **中间结果常驻 Reg**, 仅在进入和退出这段计算时与UB交互一次
- 显著减少数据搬运, 计算链越长收益越大
- 适合长计算链 (如 Gelu/LayerNorm/Softmax 等)



样例地址

https://gitcode.com/cann/asc-devkit/tree/master/examples/01_simd_cpp_api/05_best_practices/02_reg_compute/gelu_high_performance

02 高阶性能打磨 ② VF循环展开

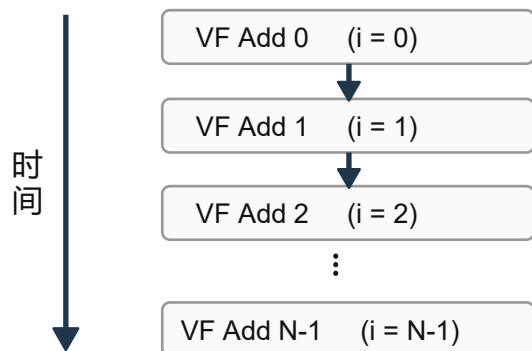
VF 循环展开

VF 函数内的 for 循环为简单迭代结构，编译器默认按序生成循环控制指令（比较、跳转），导致循环迭代之间存在标量开销。

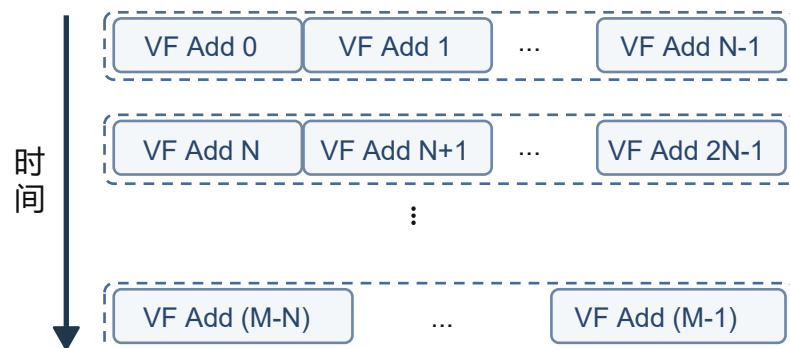
通过 **#pragma unroll N** 告诉编译器将循环展开 N 份，消除循环控制开销，同时使多组迭代的 VF 指令可交错排布，提高指令级并行度（ILP），使更多指令能够连续双发射。

✦ Reg编程支持 VF 双发特性，常规计算指令并行度可达 512 bytes/cycle，具体信息可以查看手册 [1]。

指令排布（串行）



指令排布（并行）



02 高阶性能打磨 ② VF循环展开

Unroll 前：顺序执行

```
for (i = 0; i < len; i++) {  
    AscendC::Reg:Mul(yReg, xReg, xReg, mask);  
    .....  
}
```

未展开：

Iter0: Load→Mul→...→Store

↓ (循环控制开销)

Iter1: Load→Mul→...→Store

↓ (循环控制开销)

Iter2: Load→Mul→...→Store

aiv_vec_time
66.277 μs



循环控制开销存在，指令并行度受限

Unroll 后：指令并行发射

```
#pragma unroll N  
for (i = 0; i < len; i++) {  
    AscendC::Reg:Mul(yReg, xReg, xReg, mask);  
    .....  
}
```

展开后：

Iter0: Load→Mul→...→Store

Iter1: Load→Mul→...→Store

Iter2: Load→Mul→...→Store

Iter3: Load→Mul→...→Store

... (无循环控制开销，指令可交错排布)

aiv_vec_time
63.655 μs



消除循环控制开销，提高指令级并行



#pragma unroll N 展开 VF 循环。



消除循环控制开销，
提高指令级并行。



展开因子需要调优，
过大会增加寄存器压力。



验证结果（已核验数据）



IPC **1.20 -> 1.25**



向量耗时再减约 **4.6%**



性能数据：

	Task Duration	aiv_time	aiv_vec_time	aiv_vec_ratio	aiv_scalar_time	aiv_scalar_ratio	aiv_mte2_time	aiv_mte2_ratio	aiv_mte3_time	aiv_mte3_ratio
Case1	348.868	348.826	66.277	0.19	3.03	0.009	320.543	0.919	314.547	0.902
Case2	344.436	344.081	63.655	0.185	4.757	0.014	315.468	0.917	306.069	0.900

目录

Part 1 昇腾矢量计算基础

Part 2 矢量算子编程实践

2.1 基础矢量编程入门

2.2 MemBase中高阶打磨

2.3 RegBase 中高阶打磨

Part 3 能力全景与开发路径

03 能力全景与开发路径 手册 + 样例：你的两个工具箱

 **手册 docs/**
讲原理、流程和 API

 **原理与架构**
AI Core、SIMD、存储层级
(GM / UB / Reg / VF)

 **开发流程**
Host → Device → AI Core
编译、运行与调试

 **API 参考**
Ascend C API 详解
DataCopy、PipeBarrier 等

 **API 速查**
SIMD API 速查手册
(docs/api/SIMD-API/)

目录示例

```
asc-devkit/docs
├── api/
├── guide/
│   ├── 入门教程
│   ├── 编程指南
│   ├── 算子实践参考
│   └── 兼容性迁移指南
│   .....
└── .....
```

 **样例 examples/**
给可编译、可运行的完整工程

 **完整工程**
可直接编译、运行
快速验证思路

 **典型算子**
Add、Gelu、MatMul 等
覆盖常见场景

 **性能优化样例**
多核、双缓冲、Reg/VF
融合等进阶实践

 **对照学习**
配套 README 与注释
便于理解与对照

目录示例

```
asc-devkit/examples/01_simd_cpp_api
├── 00_introduction
├── 01_utilities
├── 02_features
├── 03_basic_api
├── 04_advanced_api
├── 05_best_practices
├── 06_compatibility_guide
└── .....
```

✓ 手册 docs/ 讲原理、流程、API 以及实践指南。

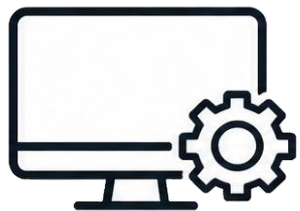
✓ 样例 examples/ 给可编译、可运行的完整工程。

 手册地址: <https://asc.gitcode.com/>

 样例地址: <https://gitcode.com/cann/asc-devkit/tree/master/examples>

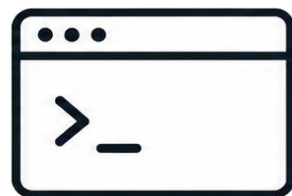
03 能力全景与开发路径

1. 环境准备



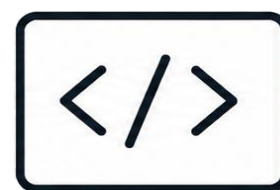
- 1 安装 CANN
- 2 source set_env.sh

2. 跑通 Add 样例



- 1 Cmake & make
- 2 gen_data
- 3 ./demo
- 4 verify

3. 改造成自己的算子



- 1 换计算 API
- 2 改 shape
- 3 按瓶颈优化

结合手册与样例，从能跑开始，再逐步追性能。



手册地址：

<https://asc.gitcode.com>



样例地址：

<https://gitcode.com/cann/asc-devkit/blob/master/examples>

Thank you.



Ascend C 算子编程交流群

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯